Sarita Sangvez 21st October 2025

23i-2088

CY-A

Assignment 02

Q1.

a) 1. Merge sort always runs in $O(n \log n)$ while Quick sort can give $O(n^2)$ in worst case

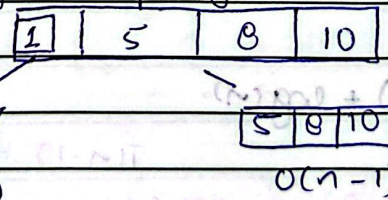
2. Merge sort is stable sort whereas Quick sort may not be.

3. Merge sort uses divide and conquer which makes it preferable for large datasets.

4. Merge sort works better for linked lists as it does not require random access like Quick sort

b) Quick sort performs poorly when pivot selection is unbalanced

e.g



- Input may be already reverse sorted
- Pivot is always smallest or largest.

$T(n) = T(n-1) + n = n^2$

c) Input may be nearly sorted.

Insertion sort's avg and worst time is $O(n^2)$

and best case (if array is sorted) is $O(n)$

Less comparison and swaps needed.

Date

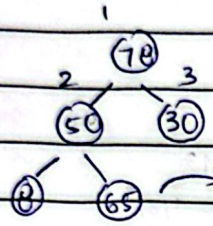
a) Insertion sort - It is efficient for nearly sorted data

$T(n) = O(n + k^2)$ → almost negligible so linear time

If array is large and we want a fixed performance - $O(n \log k)$, we use heap sort.

Q 2.

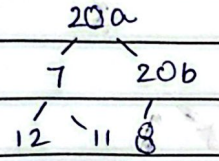
a)

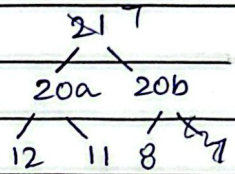


→ violates max heap property

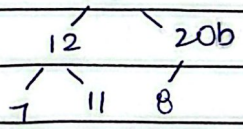
b) 1) [7 20a 20b 12 11 8 | 21]

Heapify :



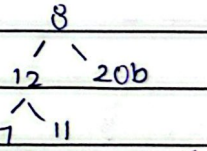


Heapify :

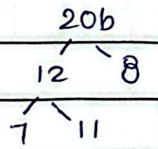


[20a 12 20b 7 11 8 | 21]

2) [8 12 20b 7 11 | 20a 21]



heapify :



[20b 12 8 7 11 | 20a 21]

3) 11 12 8 7 | 20b 20a 21]

here, 20b appears first in sorted array. If we compare using >=, order may change so Heap sort is NOT stable.

c) At height h, there are at most $\frac{n}{2^{h+1}}$

Heapify takes O(h) as we are just swapping

$T = \underbrace{k}_{\text{height at one level}} O(h)$

$T = \frac{n}{2^{h+1}} O(h)$

Date

We know $\sum_{n=0}^{n=\infty} nx^n = \frac{x}{(1-x)^2}$

So, $\sum_{n=0}^{n=\log n} \frac{n}{2^{n+1}} O(n) = \sum_{n=0}^{n=\log n} \frac{n \cdot n}{2^n \cdot n^1}$

$= n \sum_{0}^{n=\log n} \frac{n}{2^n \cdot n^1}$

$= n \sum_{n=0}^{n=\infty} \frac{1}{2} \cdot n \left(\frac{1}{2}\right)^n$, $x = 1/2$

$= \frac{n}{2} \sum_{n=0}^{n=\infty} n x^n = \frac{n}{2} \cdot \frac{1/2}{(1-1/2)^2} = \frac{n}{2} [2]$

$= n$

Q3.

a) function swapEnds (S) {

first ← S.removeFromFront() // remove 1st element

last ← S.removeFromLast() // remove last element

S.addToFront(last) // put last at front

S.addToBack(first) } // put first at last

b) function shiftLeft (S, k) {

for i ← 1 to k {

temp ← S.removeFromFront() // take 1st element

S.addToBack(temp) // move to end

}

}

Q4.

a) A = [(r1, s1,) (r2, s2) ...]

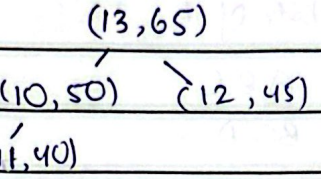
Need k highest scores.

Date

we can use Max Heap.

Building heap takes O(|A|) time and extracting top k elements takes O(k log|A|)

e.g A = [(10, 50) (11, 40) (12, 45) (13, 65)]



extract top 2 elements :

1. (11,40) heapify : 10,50 | 13
(10,50) (12,45) 11,40 12,45

2. (12,45) [10, 13] | 10 13
(11,40) returned top 2 gardens.

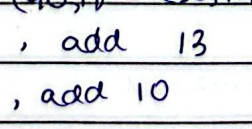
b) 1. we start at root. if root's score is less than x, we stop.

2. we traverse :

For a node (s, v), if s > x, add v to result, then check both children as they will be > x

If s < x, donot visit children. (they can't be > x)

Recursive approach e.g DFS

e.g 

and x = 42.

1. 65 > 42, add 13
2. 50 > 42, add 10
3. 40 ≤ 42, stop
4. 30 ≤ 14, stop
5. 45 > 42, include 12

Q5.

1. Text [1..n] of length n
Pattern [1..m] of length m.

Date

→ Best case happens when a mismatch occurs on the 1st character of each shift.

e.g Text T = ABCDABCDABCD

Pattern P = ZBC , n = 12 , m = 3.

Shift	substr of T	comparison	
0	ABC	A ≠ Z	no match
1	BCD	B ≠ Z	no match
2	CDA	C ≠ Z	no match

So total comparisons : n − m + 1

O(n)

→ worst case happens when pattern and text have alot of common prefixes so algo keeps making partial matches.

e.g T = AAAAAAAAAA , P = AAAAA

In 5 shifts, it will make 5 comparisons in each shift and get a match.

Total shifts = n − m + 1 = 6

each shift does 5 (m) comparisons.

So m · (n − m + 1) = O(nm)

2. To impove the algorithm, we can skip unnecessary shifts that recheck characters that have already been checked we should keep content of the pattern. (KMP)

- It will not be feasible for all cases e.g
  - Text is too short
  - The pattern rarely matches
  - It also creates extra space as LPS is computed.

Code :

```
void LPS (string pat, int m, int lps[]) {
    int j = 1
    lps[0] = 0
```

Date

```
int i = 0 ;
while (j < m) {
    if ( pat[i] == pat[j]) {      // match found
        i++ ;
        lps[j] = i ;
        j++ ; }
    else {                        // not a match
        if (i != 0)
            i = lps[i-1] ; // use content.
        else {                    // i is 0
            lps[j] = 0 ;
            j++ ; }
        }
    return lps }

KMP ( string text , string pat) {
    int n = text.length() ;
    int m = pat.length() ;
    int lps[m] ;
    (pat, m, lps);
    int i = 0 , j = 0 ;
    while (j < n) {
        if (text[i] == text[j]) { i++ ; j++ ; }

        if (i == m) {
            cout << "pattern found" ;
            i = lps[i-1] ; } // traverse rest of are
        else if (j < n && text[i] != pat[j])
            if (i != 0)
                i = lps[i-1] ;
            else j++ ;
        }
    }
}
```

Kaaghaz Qalam

Date

Q6.

a)

	0	1	2	3	4	5	6	7	8
	a	a	b	a	a	b	c	a	b
	0	1	0	1	2	3	0	1	2

b)

```
int main() {
    string pat = "aabaabcb";
    int lps[pat.length()];
    cout << "LPS array : ";
    // call function.
    LPS( pat, p.length(), lps );

    for (int i=0; i<pat.length(); ++i)
    { cout << lps[i] << " "; }

    return 0; }
```

code :

c)

```
void LPS (string pat, int m, int eps[]) {
    int j = 1
    eps[0] = 0
```

Date

```
int i = 0 ;
while (j < m) {
    if ( pat[i] == pat[j]) {      // match found
        i++ ;
        lps[j] = i ;
        j++ ; }
    else {                        // not a match
        if (i != 0)
            i = lps[i-1] ;        // use context.
        else {                    // i is 0
            lps[j] = 0 ;
            j++ ; }
    }
return lps }
```

Q7.

a) Text T = 3 1 4 1 5 9 2 6 5 3 5 8 9 7 9 3 (length = 16)

Pattern P = "26" length (m = 2)

base d = 10 (10 digits possible), MOD q = 11.

General: $hash(P) = (P[0] \times d^{m-1} + P[1] \times d^{m-2} \dots P[m-1]) \bmod q$

$hash(26) = (2 \times 10^1 + 6 \times 10^0) \bmod 11 = 4$

we use: $hash(ab) = (10a + b) \bmod q$

$newHash = \left(d \times (oldHash - T[i] \times d^{m-1}) + T[i+m] \times d^0 \right) \bmod q$

$= \left((oldHash - T[i] \cdot 10^1) \times 10 + T[i+1] \right) \bmod 11$

leaving window — entering window.

All length 2 windows. $n - m + 1 = 15$ windows.

1. $w_1 = (3 \times 10 + 1) \bmod q = 9.$

2. $w_2 = 14 = \left((9 - 3 \times 10^1) \times 10 + 4 \right) \bmod 11$

$= (-206) \bmod 11 \Rightarrow (11 - (206 \bmod 11))$

$= 11 - 8 = 3.$

Kaaghaz Qalam

Date remove 1, add 1

3. $W_3 = 41 = [(3 - 1 \times 10) \times 10 + 1] \bmod 11$

$= (11 - (69 \bmod 11)) = 8$

we can do direct calculation.

4. $W_4 = 15 = 15 \bmod 11 = 4$ (hash match)

5. $W_5 = 59 = 59 \bmod 11 = 4$ (hash match)

7. $W_7 = 26 \bmod 11$ (hash match)

8. $65 \bmod 11 = 10$

9. $53 \bmod 11 = 9$

10. $35 \bmod 11 = 2$

11. $58 \bmod 11 = 3$

6. $W_6 = 92 \bmod 11 = 4$ (hash match)

12. $89 \bmod 11 = 1$

13. $97 \bmod 11 = 9$

14. $79 \bmod 11 = 2$

15. $93 \bmod 11 = 5$.

hash matched = 4.

Actual pattern = 1. (window 7).

Spurious Hits = 4 - 1 = 3

- we worked with a small modulo so we got more hash collisions. Since Rabin Karp relies on verifying the actual substring when hash matches, these became extra work.

b)

```
int main() {
    string text = "3141592653589793";
    string pattern = "26";
    int q = 11;  int d = 10;  int n = text.length(); int m
    = pattern.length();
    int pHash = 0;  // hash of pattern
    int tHash = 0;  int h = 1;  // d^(m-1)
        // compute h = d^(m-1) % q
    for (int i = 0; i < m-1; ++i) {
    if h = (h * d) % q; }
```

Date

// compute initial hashes.

for (int i = 0; i < m ; ++i) { → convert to its actual integer value

'26' pHash = (d^10 * pHash^2 + (pattern^26[i] - '0')) % q^11

tHash = (d * tHash + (text[i] - '0')) % q }

int sp=0 ; int matches = 0

// slide over text

for (int i = 0 ; i <= n - m ; ++i) {

if (pHash = tHash) {

// check chars

if (text.substr(i,m) == pattern) {

cout << "Pattern found at: " << i << endl;

matches ++ ; }

else {

cout << " spurious Hit at : " << i << endl;

sp++ ;

}

// compute next window.

if (i < n - m) { oldhash leaving $d^{m-1} \bmod q$ add

tHash = (d * (tHash - (text[i] - '0') * h) + (text[i+m] - '0')) % q ;

if (tHash < 0)

{ tHash += q; }

}

}

return 0;

}

Q 8. A 2D text array $n \times n$

A 2D pattern $m \times m$ $m \leq n$.

We need to find locations in the text where pattern exactly matches a sub array of $m \times m$.

Kaaghaz Qalam

Date

- We will compute a hash for 2D block size: $m \times m$
- Slide window horizontally and vertically.

2D hash function can be: e.g $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$

$H(P) = a \times 10^3 + b \times 10^2 + c \times 10^1 + d \times 10^0 = 10890$

$H(P) = (Row1Hash \times 10 + Row2Hash) \bmod 11$

$$H(P) = \sum_{i=0}^{m-1} \sum_{j=0}^{m-1} P[i][j] \times d^{(m-1-i)+(m-1-j)}$$

where i = row index, j = col. index,

$P[i][j]$ = ascii character, $d^{\wedge\{(m-1-i)+(m-1-j)\}}$ gives each cell a unique weight depending on its row and column position.

- When sliding the block right, update hash by subtracting contribution of leftmost column and add new rightmost column.
- When moving down, update by removing top row and adding new bottom row.

Update is done in $O(m)$ time.
Initial hashes calculation done in $O(m^2)$ time
Sliding horizontally and vertically done in $O((n-m+1)^2)$
Total time taken: $O(n^2)$.

Q 9.

Part a:

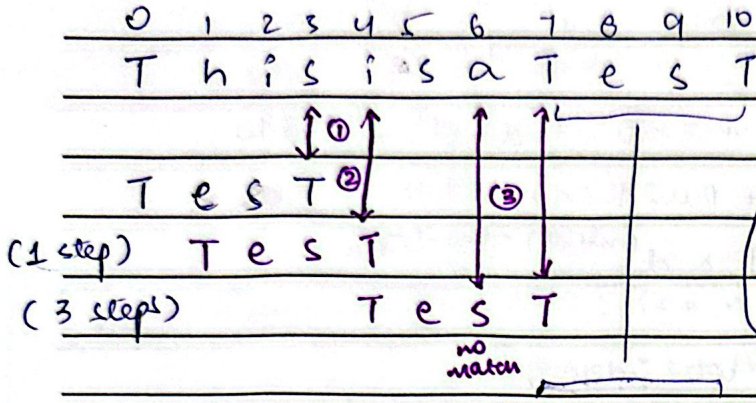
Boyer – Moore Algorithm uses 2 rules to skip characters:
1. Bad character Rule 2. Good Suffix Rule.

→ Compare from left to right
- If last character matches, move left.
- If mismatch, shift the whole pattern such that mismatched text character lines up with the last occurence of that character in pattern.
- If character does not appear in the pattern, skip the entire pattern length.

Date

e.g Text : T h i s i s a T e s T

Pattern : T e s T



Bad Match Table:

0	1	2	3
T	e	S	T	*
3 1	2	1	1	3

- Assign value to each char use formula : max

max (1, |p| - index - 1)

1. max (1, 4 - 0 - 1) = 3

max (1, 4 - 3 - 1) = 1

pattern present in 7th index.

Two 'T' in pattern so we take latest 'T'.

* → no. of non-repeating char. (for characters not present in bad match table)

Part b :

```
// pre-processing function
void badchar ( const string & str , int n , int badchar[256])
{
for (int i = 0 ; i < 256 ; ++i)
    badchar[i] = -1      // initialize
// value fill of last occurence of a char
for (int i=0 ; i < n ; ++i)
    badchar [ (int) str[i] ] = i ;
}

// Boye moore algo function
void BM ( const string & text , const string & pat)
{
    int m = pat . size ( ) ;
    int n = text . size ( ) ;
```

Date

```
int badchar[256];
    // fill badchar array
badchar(pat, m, badchar);
    int s;   // shift of pattern
    while (s <= (n - m))
    { int j = m - 1
    // keep decreasing j if char matching
        while (j >= 0 && pat[j] == text[s+j])
            j--;
    // if pattern is present at shift s, j becomes -1
        if (j < 0)
        { cout << "pattern found at shift"
            << s << endl;
// shift pattern to align        if (s+m < n) // occurs at the end of text pat
            s += m - badchar[text[s+m]];
        else
            s+=1;
        }
        else {
    // shift pattern to align
        s += max(1, j - badchar[text[s+j]]);
        }
    }
}
int main() {
    string txt = "Sarita";
    string pat = "ari";
    BM(txt, pat);
    return 0;
    }
```

Date

Part C

1. $\Sigma = \{a, b, c\}$, $Q = |P| + 1 = 8$

Pattern : a b a b a c b

1. $\delta(0, a) = \sigma(a) = 1$

   $\delta(0, b) = \sigma(b) = 0$, $\delta(0, c) = 0$.

2. $\delta(1, a) = \sigma(aa) = 1$

   $\delta(1, b) = \sigma(ab) = 2$; $\delta(1, c) = \sigma(ac) = 0$.

3. $\delta(2, a) = \sigma(aba) = 3$

   $\delta(2, b) = \sigma(abb) = 0$, $\delta(2, c) = \sigma(abc) = 0$

4. $\delta(3, a) = \sigma(abaa) = 1$

   $\delta(3, b) = \sigma(abab) = 4$, $\delta(3, c) = \sigma(abac) = 0$

5. $\delta(4, a) = \sigma(ababa) = 5$

   $\delta(4, b) = \sigma(ababb) = 0$, $\delta(4, c) = \sigma(ababc) = 0$

6. $\delta(5, a) = \sigma(ababaa) = 1$

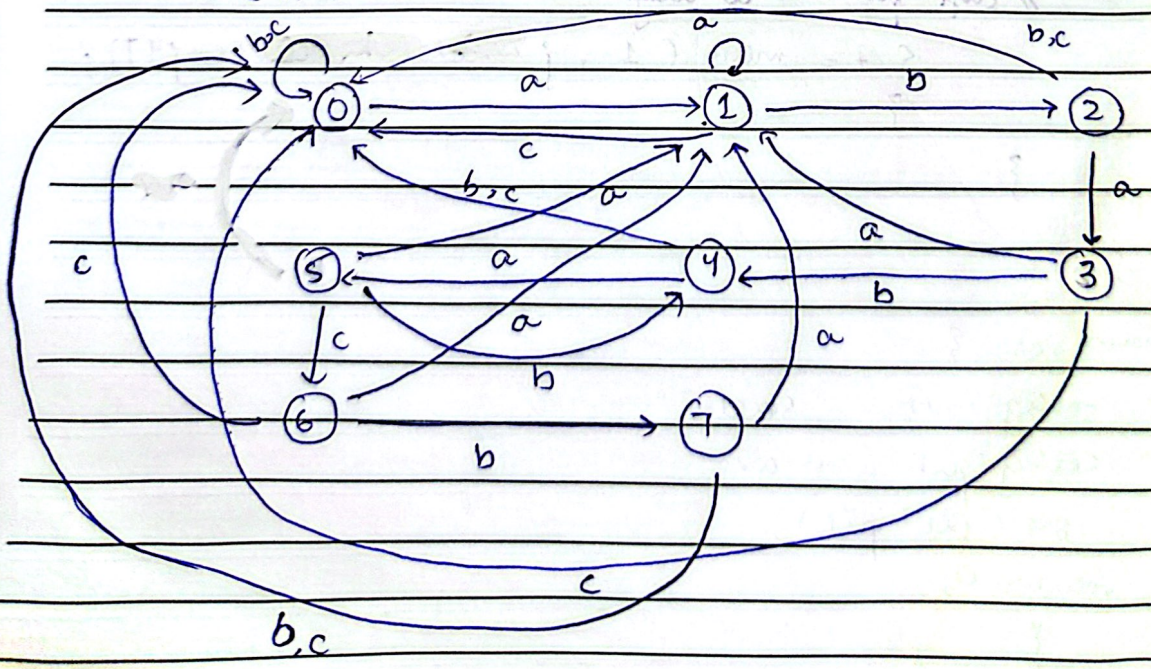
   $\delta(5, b) = \sigma(ababab) = 4$, $\delta(5, c) = \sigma(ababac) = 6$

7. $\delta(6, a) = \sigma(ababaca) = 1$

   $\delta(6, b) = \sigma(ababacb) = 7$, $\delta(6, c) = \sigma(ababacc) = 0$

8. $\delta(7, a) = \sigma(ababacba) = 1$

   $\delta(7, b) = \sigma(ababacbb) = 0$, $\delta(7, c) = 0$



Kaaghaz Qalam

Date

State Table.

State	a	b	c
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	5	0	0
5	1	4	6
6	1	7	0
7	1	0	0

Solution set:

A	State
a	1
b	2
a	3
b	4
a	5
c	6
b	7 (string matched)

2. i) Pattern: a b a b a d c a (8)

$\Sigma = \{a, b, c, d\}$ $Q = 9$.

1. $\delta(0,a) = 1$

   $\delta(0, b/c/d) = 0$.

2. $\delta(1,a) = 1$, $\delta(1,b) = 2$

   $\delta(1, d/c) = 0$

3. $\delta(2,a) = 3$,

   $\delta(2, b/c/d) = 0$,

4. $\delta(3,a) = 1$, $\delta(3, c/d) = 0$.

   $\delta(3,b) = 4$

(rough work on different notebook)

Kaaghaz Qalam

Date

State Diagram.

State	a	b	c	d
0	1	0	0	0
1	1	2	0	0
2	3	0	0	0
3	1	4	0	0
4	5	0	0	0
5	1	0	6	0
6	1	0	0	7
7	8	0	0	0
8	1	0	0	0

Solution set.

A	state
a	1
b	2
a	3
b	4
a	5
b	0
d	0
c	0
a	1
a	1
b	2
a	3
b	4
a	5
d	6
c	7
a	8 (string matched)

mismatch

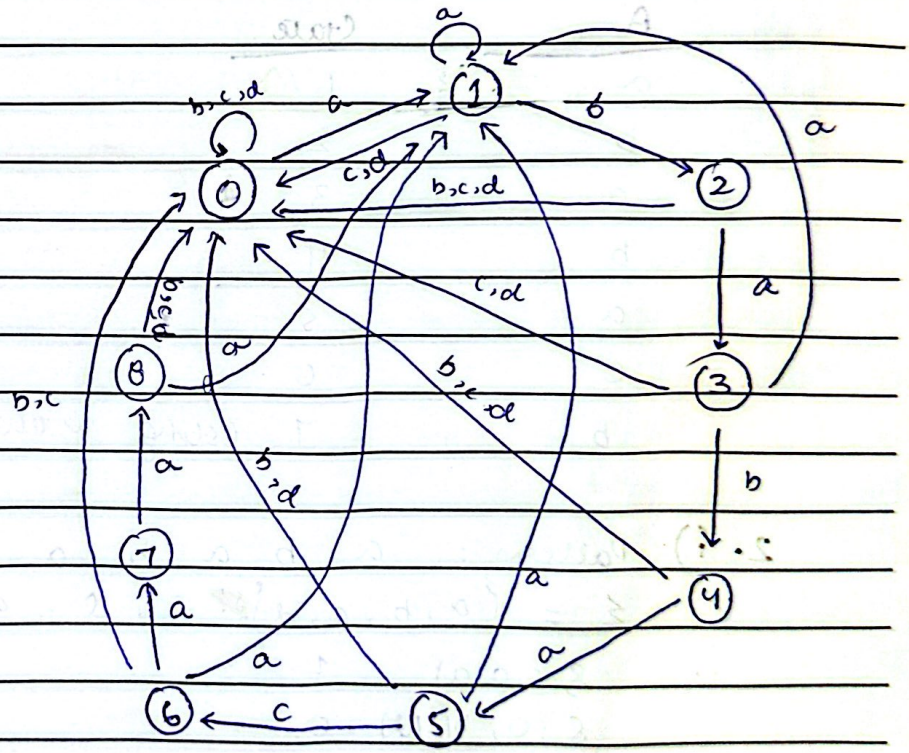
(suffix is prefix of pattern)

aa

ab

aba

abab



Kaaghaz Qalam

A = a b a b a d c a a b a b a d c a d

ii) S = a b a b a d c a

State Table

States	a	b	c	d
	(same as prev)			

Solution set.

A	state
a	1
b	2
a	3
b	4
a	5
d	6
c	7
a	8 (string matched)

A	state
a	1
b	2
a	3
b	4
a	5
d	6
c	7
a	8 (string matched)
d	0

Diagram (same as prev).

iii) A = a b a b a b d c b a b a b a b a d c b

State Table and Diagram
(same as prev)

Solution set

A	State	
a	1	
b	2	
a	3	
b	4	
a	5	
mismatch b	0	
d	0	
c	0	
b	0	This automation never reaches state B (accepting state), so string S is not a substring of A.
a	1	
b	2	
a	3	
b	4	
a	5	
d	6	
c	7	
b	0	